

Cliente Vue para API Rest con Laravel 10

Tabla de contenidos

1	Cliente en Vue.js para API REST de Laravel 10	2
1.1	El servidor	2
1.2	Instalación y Configuración del Proyecto Vue	2
1.2.1	1.1 Crear un nuevo proyecto Vue con Vite	2
1.2.2	1.2 Instalar Axios y Vuex para manejo de estado	2
1.2.3	1.3 Configurar las variables de entorno	2
1.3	Configurar App.vue	2
1.4	Configurar la Conexión con la API	3
1.4.1	2.1 Crear una instancia de Axios	3
1.5	Manejo de Autenticación con Vuex	4
1.5.1	3.1 Configurar el Store de Vuex	4
1.6	Crear Componentes para los Posts	5
1.6.1	4.1 Componente de Lista de Posts	5
1.7	Agregar Vista para Crear un Post	6
1.7.1	4.1 Crear el Componente CreatePost.vue	6
1.8	Configurar Rutas con Vue Router	8
1.8.1	5.1 Instalar Vue Router	8
1.8.2	5.2 Configurar las Rutas	8
1.9	Crear un Componente de Login	8
1.10	Ejecutar el Proyecto	9
1.11	Consideraciones para Nginx como Reverse Proxy	9
1.12	Mejoras Futuras	10
1.12.1	DONE Agregar un formulario para crear posts.	10
1.12.2	Implementar edición y eliminación de posts.	10
1.12.3	DONE Agregar middleware en rutas para restringir acceso si no está autenticado.	10
2	Usar Laravel Sail para Crear una Aplicación Front-end con Vue.js (Alternativa)	10
2.1	1. Inicia Laravel Sail	10
2.2	2. Crear la Aplicación Vue con Vite	10
2.3	3. Navegar al Directorio del Cliente Vue	10
2.4	4. Instalar Dependencias	11
2.5	5. Configurar el Proxy en Vite	11
2.6	6. Ejecutar la Aplicación Vue	11

2.7	Resumen de Comandos	11
3	Comparación entre Laravel Sail y Configuración Independiente	12
3.1	Usar Laravel Sail	12
3.1.1	Ventajas	12
3.1.2	Desventajas	12
3.2	Hacerlo de Forma Independiente	12
3.2.1	Ventajas	12
3.2.2	Desventajas	12
3.3	Recomendación	12

1 Cliente en Vue.js para API REST de Laravel 10

Este documento describe los pasos necesarios para crear un cliente en Vue.js que consuma la API REST de Laravel 10 con autenticación JWT.

1.1 El servidor

Primero hay que tener en cuenta que el servidor, después de hacer:

```
sail up -d
```

tenemos que hacer

```
sail artisan serve
```

1.2 Instalación y Configuración del Proyecto Vue

1.2.1 1.1 Crear un nuevo proyecto Vue con Vite

Ejecuta el siguiente comando para generar un proyecto Vue:

```
npm create vite@latest laravel-client --template vue
cd laravel-client
npm install
```

1.2.2 1.2 Instalar Axios y Vuex para manejo de estado

Axios se usará para realizar peticiones HTTP y Vuex para manejar la autenticación:

```
npm install axios vuex@4
```

1.2.3 1.3 Configurar las variables de entorno

Crea un archivo `.env` en la raíz del proyecto Vue y agrega la URL de la API Laravel:

1.3 Configurar App.vue

Configurar App.vue

Sustituye el contenido de `src/App.vue` por lo siguiente para integrar Vue Router y mostrar las rutas correctamente:

```

<template>
  <div>
    <nav>
      <router-link to="/">Home</router-link> |
      <router-link to="/login">Login</router-link>
      <router-link to="/create-post">Crear Post</router-link>
    </nav>
    <router-view />
  </div>
</template>

<script>
export default {
  name: "App",
};
</script>

<style>
nav {
  margin-bottom: 20px;
}
nav a {
  margin-right: 10px;
}
</style>

```

1.4 Configurar la Conexión con la API

1.4.1 2.1 Crear una instancia de Axios

Crea un archivo `src/services/api.js`:

```

import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  }
});

export default api;

```

1.5 Manejo de Autenticación con Vuex

1.5.1 3.1 Configurar el Store de Vuex

Crea un archivo `src/store/index.js`:

```
import { createStore } from 'vuex';
import api from '@services/api';

export default createStore({
  state: {
    user: null,
    token: localStorage.getItem('token') || null
  },
  mutations: {
    setUser(state, user) {
      state.user = user;
    },
    setToken(state, token) {
      state.token = token;
      localStorage.setItem('token', token);
      api.defaults.headers['Authorization'] = `Bearer ${token}`;
    },
    logout(state) {
      state.user = null;
      state.token = null;
      localStorage.removeItem('token');
      delete api.defaults.headers['Authorization'];
    }
  },
  actions: {
    async login({ commit }, credentials) {
      const response = await api.post('/auth/login', credentials);
      commit('setToken', response.data.access_token);
      commit('setUser', await api.post('/auth/me'));
    },
    async logout({ commit }) {
      await api.post('/auth/logout');
      commit('logout');
    },
    async fetchUser({ commit }) {
      const response = await api.post('/auth/me');
      commit('setUser', response.data);
    }
  }
});
```

1.6 Crear Componentes para los Posts

1.6.1 4.1 Componente de Lista de Posts

Creando src/components/PostList.vue:

```
<template>
  <div>
    <h1>Lista de Posts</h1>
    <transition name="fade">
      <div v-if="flashMessage" class="flash-message" @click="flashMessage = ''">
        {{ flashMessage }}
      </div>
    </transition>
    <ul>
      <li v-for="post in posts" :key="post.id">
        <h3>{{ post.title }}</h3>
        <p>{{ post.description }}</p>
        
    </ul>
  </div>
</template>

<script>
import api from '@services/api';

export default {
  data() {
    return {
      posts: [],
      flashMessage: localStorage.getItem('flashMessage') || ''
    };
  },
  async mounted() {
    try {
      const response = await api.get('/posts');
      this.posts = response.data.data;
      if (this.flashMessage) {
        setTimeout(() => {
          this.flashMessage = '';
          localStorage.removeItem('flashMessage');
        }, 3000);
      }
    } catch (error) {
      console.error('Error al cargar los posts', error);
    }
  }
}
```

```

    }
  };
</script>

<style>
.post-image {
  max-width: 100%;
  height: auto;
  margin-top: 10px;
}

.flash-message {
  background-color: #4caf50;
  color: white;
  padding: 10px;
  margin-bottom: 15px;
  text-align: center;
  border-radius: 5px;
  cursor: pointer;
}

.fade-enter-active, .fade-leave-active {
  transition: opacity 0.5s;
}
.fade-enter, .fade-leave-to {
  opacity: 0;
}
</style>

```

1.7 Agregar Vista para Crear un Post

1.7.1 4.1 Crear el Componente CreatePost.vue

Crea un archivo src/components/CreatePost.vue:

```

<template>
  <div>
    <h1>Crear Nuevo Post</h1>
    <form @submit.prevent="submitPost" enctype="multipart/form-data">
      <input v-model="title" type="text" placeholder="Título" required />
      <textarea v-model="description" placeholder="Descripción" required />
      <input type="file" @change="handleFileUpload" required />
      <button type="submit">Publicar</button>
      <p v-if="message" :class="{ 'error': isError, 'success': !isError}">{{ message }}
    </form>
  </div>
</template>

```

```

</template>

<script>
import api from '@services/api';

export default {
  data() {
    return {
      title: '',
      description: '',
      image: null,
      message: '',
      isError: false
    };
  },
  methods: {
    handleFileUpload(event) {
      this.image = event.target.files[0];
    },
    async submitPost() {
      try {
        const formData = new FormData();
        formData.append('title', this.title);
        formData.append('description', this.description);
        formData.append('image', this.image);

        const response = await api.post('/posts', formData, {
          headers: {
            'Content-Type': 'multipart/form-data'
          }
        });

        localStorage.setItem('flashMessage', response.data.message || 'Post creado correctamente');
        this.$router.push('/');
      } catch (error) {
        console.error('Error al crear el post', error);
      }
    }
  },
};
</script>

<style>
.error {
  color: red;

```

```
}  
.success {  
  color: green;  
}  
</style>
```

1.8 Configurar Rutas con Vue Router

1.8.1 5.1 Instalar Vue Router

```
npm install vue-router@4
```

1.8.2 5.2 Configurar las Rutas

Crea el archivo `src/router/index.js`:

```
import { createRouter, createWebHistory } from 'vue-router';  
import PostList from '@/components/PostList.vue';  
import Login from '@/components/Login.vue';  
import CreatePost from '@/components/CreatePost.vue';  
  
const routes = [  
  { path: '/', component: PostList },  
  { path: '/login', component: Login },  
  { path: '/create-post', component: CreatePost }  
];  
  
const router = createRouter({  
  history: createWebHistory(),  
  routes,  
});  
  
export default router;
```

Modifica `src/main.js` para usar Vue Router y Vuex:

```
import { createApp } from 'vue';  
import App from './App.vue';  
import router from './router';  
import store from './store';  
  
const app = createApp(App);  
app.use(router);  
app.use(store);  
app.mount('#app');
```

1.9 Crear un Componente de Login

Crea `src/components/Login.vue`:


```

<template>
  <div>
    <h1>Iniciar Sesión</h1>
    <form @submit.prevent="handleLogin">
      <input v-model="email" type="email" placeholder="Correo" required>
      <input v-model="password" type="password" placeholder="Contraseña" required>
      <button type="submit">Ingresar</button>
    </form>
  </div>
</template>

<script>
import { mapActions } from 'vuex';

export default {
  data() {
    return {
      email: '',
      password: ''
    };
  },
  methods: {
    ...mapActions(['login']),
    async handleLogin() {
      try {
        await this.login({ email: this.email, password: this.password });
        this.$router.push('/');
      } catch (error) {
        console.error('Error en el login', error);
      }
    }
  }
};
</script>

```

1.10 Ejecutar el Proyecto

Para iniciar el servidor Vue, ejecuta:

```
npm run dev
```

Esto iniciará el frontend en `http://localhost:5174`. Y el 80 para producción.

1.11 Consideraciones para Nginx como Reverse Proxy

Si el frontend Vue está en una máquina separada de Laravel detrás de Nginx, asegúrate de que la configuración de Nginx permite solicitudes CORS correctamente. Agrega esta configuración al

bloque del servidor en nginx.conf

```
location /api/ {
    proxy_pass http://backend-server:8000/api/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Si el frontend y backend están en el mismo servidor, asegúrate de que el reverse proxy esté configurado para servir ambos correctamente.

Con esta actualización, el cliente Vue.js puede interactuar con el backend Laravel detrás de Nginx sin problemas.

1.12 Mejoras Futuras

1.12.1 **DONE** Agregar un formulario para crear posts.

1.12.2 Implementar edición y eliminación de posts.

1.12.3 **DONE** Agregar middleware en rutas para restringir acceso si no está autenticado.

—

2 Usar Laravel Sail para Crear una Aplicación Front-end con Vue.js (Alternativa)

Puedes usar Laravel Sail para crear tu aplicación front-end con Vue.js. Aquí te dejo los pasos que puedes seguir para hacerlo:

2.1 1. Inicia Laravel Sail

Asegúrate de que tu aplicación Laravel Sail esté en funcionamiento:

```
./vendor/bin/sail up -d
```

2.2 2. Crear la Aplicación Vue con Vite

Utiliza el contenedor de Laravel Sail para crear la aplicación Vue.js usando Vite:

```
./vendor/bin/sail npm create vite@latest laravel-client --template vue
```

2.3 3. Navegar al Directorio del Cliente Vue

Después de crear la aplicación Vue, navega al directorio del proyecto:

```
cd laravel-client
```

2.4 4. Instalar Dependencias

Instala las dependencias necesarias utilizando npm dentro del contenedor de Sail:

```
./vendor/bin/sail npm install
```

2.5 5. Configurar el Proxy en Vite

Para que Vite pueda comunicarse con tu API Laravel, configura un proxy en vite.config.js:

```
export default {  
  server: {  
    proxy: {  
      '/api': 'http://localhost', // o la dirección donde se ejecuta tu aplicació  
    },  
  },  
};
```

2.6 6. Ejecutar la Aplicación Vue

Para ejecutar la aplicación Vue dentro del contenedor de Sail, usa el siguiente comando:

```
./vendor/bin/sail npm run dev
```

Esto debería iniciar el servidor de desarrollo de Vite y permitirte ver tu aplicación Vue en tu navegador.

2.7 Resumen de Comandos

```
# Iniciar Laravel Sail  
./vendor/bin/sail up -d  
  
# Crear la aplicación Vue  
./vendor/bin/sail npm create vite@latest laravel-client --template vue  
  
# Navegar al directorio del cliente Vue  
cd laravel-client  
  
# Instalar dependencias  
./vendor/bin/sail npm install  
  
# Configurar proxy en vite.config.js  
# (Editar el archivo vite.config.js y agregar el proxy)  
  
# Ejecutar la aplicación Vue  
./vendor/bin/sail npm run dev
```

Siguiendo estos pasos, podrás integrar una aplicación front-end Vue.js con tu API REST Laravel utilizando Sail.

—

3 Comparación entre Laravel Sail y Configuración Independiente

Ambas opciones tienen sus ventajas y desventajas. Aquí te dejo algunos puntos a considerar para cada una:

3.1 Usar Laravel Sail

3.1.1 Ventajas

- **Entorno Consistente:** Sail proporciona un entorno de desarrollo consistente utilizando Docker, lo que puede reducir problemas de configuración entre diferentes entornos de desarrollo.
- **Simplicidad:** Configurar y administrar servicios como MySQL, Redis, y otros es más sencillo con Sail, ya que todo está preconfigurado.
- **Facilidad de Uso:** Puedes aprovechar comandos simples de Sail para ejecutar tareas comunes como la instalación de dependencias, ejecutar pruebas, y más.

3.1.2 Desventajas

- **Restricciones de Docker:** Algunas veces, trabajar con Docker puede introducir problemas específicos de contenedores, como temas de rendimiento y configuración de red.
- **Curva de Aprendizaje:** Si no estás familiarizado con Docker, puede haber una pequeña curva de aprendizaje inicial.

3.2 Hacerlo de Forma Independiente

3.2.1 Ventajas

- **Flexibilidad:** Puedes personalizar tu entorno de desarrollo según tus necesidades específicas sin las restricciones de un contenedor Docker.
- **Rendimiento:** En algunos casos, un entorno no acoplado a Docker puede tener un mejor rendimiento, especialmente en sistemas de archivos y acceso a red.

3.2.2 Desventajas

- **Configuración Manual:** Necesitarás configurar manualmente servicios como bases de datos, colas, etc., lo que puede ser tedioso y propenso a errores.
- **Inconsistencias:** Podrías encontrarte con problemas de consistencia entre diferentes entornos de desarrollo y producción si no tienes cuidado.

3.3 Recomendación

Si ya estás utilizando Sail para tu API con Laravel, podría ser conveniente seguir utilizando Sail para crear y gestionar tu aplicación Vue, ya que te proporcionará un entorno de desarrollo consistente y simplificará la configuración inicial. Usar Sail también facilita la administración de dependencias y la integración continua.

Por otro lado, si prefieres un enfoque más flexible y personalizado, o si ya tienes experiencia configurando entornos de desarrollo sin Docker, podrías optar por crear la aplicación Vue de forma independiente.

En resumen:

- **Si la simplicidad y la consistencia son importantes para ti**, sigue con Sail.
- **Si necesitas más flexibilidad y rendimiento**, considera hacerlo de forma independiente.